Computer Vision

Homework 3

Francesco Setti

Fall 2025

Exercise 1 - Image rectification

The goal of this assignment is to understand the concept of image rectification, its applications, and how it is implemented using computational techniques.

Image rectification is the process of transforming an image to correct distortions, such as perspective or lens distortions, and aligning it to a specific plane or coordinate system. This process ensures that features in the image, such as lines or points, are geometrically accurate and consistent. Rectification is widely used in computer vision tasks like stereo vision, photogrammetry, and 3D reconstruction.

The main steps for Image Rectification are:

1. **Camera Calibration**: Estimate intrinsic and extrinsic camera parameters
2. **Feature Detection**: Identify key points, such as corners or edges, in the image
3. **Homography Computation**: Estimate the transformation matrix that maps points in the distorted image to a corrected plane
4. **Warping and Resampling**: Apply the transformation to adjust the image while preserving quality.

To successfully complete this exercise you have to write a python script named `rectify.py` that can be executed from command line via

```
python rectify.py myimage.jpg
```

and performs the following operations:

- Load the image `myimage.jpg` representing a sudoku puzzle as in the provided example image `sudoku.jpg` (optional: set a default value that, if no image is passed in the command line loads the example image from the local folder.)
- Extract the corners in the image

- Estimate the homography matrix by matching extracted points with the coordinates of the corners in a synthetic image plane (*i.e.* frontal undistorted view)
- Visualize the warped image using the homography matrix

NOTE: For your tests, you can use the provided image or capture one using a camera.

Optional: document the code by visualizing intermediate steps.

Exercise 2 - Image Stitching

In this exercise, you will learn the fundamentals of image stitching by creating a panorama from a set of overlapping images. You will use feature detection, matching, and homography to align and blend the images into a seamless panorama.

Given two or more overlapping images, your task is to write a Python program that stitches them together into a single panoramic image.

To successfully complete this exercise you have to write a python script named `stitch.py` that can be executed from command line via

```
python stitch.py myimage1.jpg myimage2.jpg myimage3.jpg ...
```

and performs the following operations:

- Load the images `myimageN.jpg` as in the provided example images `img0.jpg` and `img1.jpg`
- Extract the feature points from the pair of images and match them
- Estimate the homography matrix from matched points (optional: use RANSAC to achieve better results)
- Use the homography matrix to warp one image and align it with the other. Then merge the images into a single canvas, ensuring proper alignment
- Blend the images seamlessly using methods like linear blending to avoid visible seams
- Visualize the panorama image.

HINT: iteratively load one image at a time and stitch it with the output of the previous iteration.

NOTE: you can assume the images are provided in ordered way.

NOTE: the code must be able to handle at least two images. Higher grade will be assigned to code that can handle a variable number of images.

Exercise 3 - Image Impainting

Image inpainting refers to the process of restoring missing or damaged parts of an image in a visually plausible way. This technique is widely used in digital image restoration, such as repairing old photographs, removing unwanted objects, or filling gaps in visual data.

To successfully complete this exercise you have to write a python script named `inpaint.py` that can be executed from command line via

```
python inpaint.py myimage.jpg mylogo.jpg
```

and performs the following operations:

- Load the images `myimage.jpg` and `mylogo.jpg`. Use provided `pitch.jpg` and `logo.png` (optional: you can change the logo)
- Extract the feature points (corners) from `pitch.jpg`
- Estimate the homography matrix by matching extracted points with the coordinates of the corners in a synthetic image plane (*i.e.* frontal undistorted view)
- Use the homography matrix to warp the image on the frontal plane
- Blend the `mylogo` image on the central circle seamlessly using methods like linear blending to avoid visible seams
- Use the inverse of the homography matrix to warp the image back to the original view
- Visualize the final inpainted image.

Optional: use `logo2.jpg` with alpha channel, *i.e.* transparency.